

GITTY - Ai

AI-Powered Repository Knowledge Graph Platform

1. Vision

Gitty AI transforms source code repositories into an intelligent graph representation.

Instead of treating code as plain text, the platform understands:

- **Files**
- **Classes**
- **Functions**
- **APIs**
- **Dependencies**
- **Databases**
- **Frameworks**
- **Function calls**
- **Relationships**

and stores them inside Neo4j to enable:

- **Code understanding**
- **Repository Q&A**
- **Impact analysis**
- **Dead code detection**
- **Dependency visualization**
- **AI-assisted navigation**
- **RAG over codebases**

2. Goals

Functional Goals

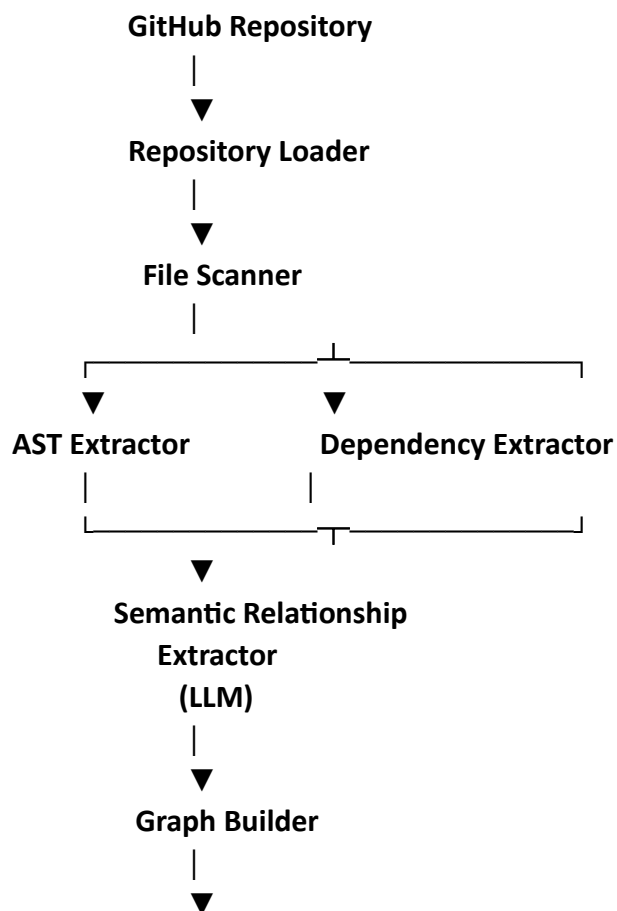
- ✓ **Read any GitHub repository**
- ✓ **Build a code knowledge graph**

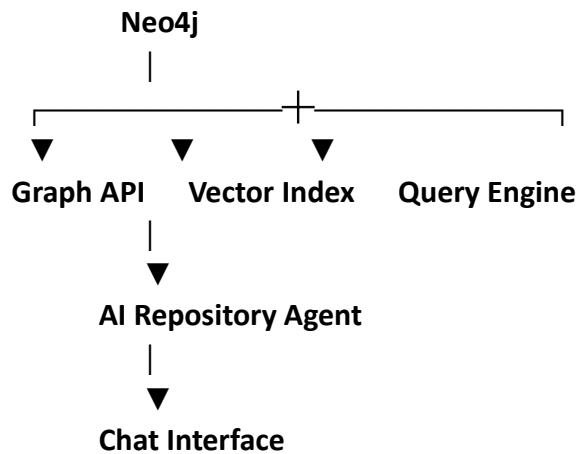
- ✓ Store graph in Neo4j
 - ✓ Support multiple languages
 - ✓ Generate embeddings
 - ✓ Answer repository questions
 - ✓ Visualize relationships
-

Non-functional Goals

- Scalable
 - Incremental updates
 - Language agnostic
 - Extensible
 - Enterprise ready
-

3. High-Level Architecture





4. System Architecture

Layer 1

Repository Ingestion

Responsible for:

- Clone repository
- Pull updates
- Branch support

Modules:

clone_repo.py

repo_manager.py

git_service.py

Layer 2

Parsing Layer

Responsible for:

- AST extraction
- Metadata extraction

Supports:

- Python
- Java

- JavaScript
- TypeScript
- Go
- C#
- C++
- PHP

Modules:

java_parser.py

python_parser.py

ts_parser.py

js_parser.py

Layer 3

Relationship Extraction

Find:

Structural

- file contains class
- class contains function

Behavioral

- function calls function

Dependency

- imports

Database

- reads from DB
- writes to DB

API

- endpoint handled by controller

Modules:

relationship_extractor.py

llm_extractor.py

Layer 4

Knowledge Graph Builder

Creates:

Nodes

Repository

Directory

File

Class

Interface

Function

Endpoint

Database

Package

Chunk

Relationships:

HAS_DIRECTORY

HAS_FILE

CONTAINS_CLASS

CONTAINS_FUNCTION

CALLS

IMPORTS

USES

DEPENDS_ON

CONNECTS_TO

READS_FROM

WRITES_TO

Modules:

graph_builder.py

neo4j_client.py

Layer 5

Embeddings Layer

Generate vectors for:

- files
- classes
- functions
- chunks

Models:

- BGE-small
- OpenAI embeddings
- Gemini embeddings

Modules:

`embedder.py`

`vector_store.py`

Layer 6

AI Agent Layer

Responsible for:

Question answering

Example:

How authentication works?

Impact analysis

Example:

What breaks if UserService changes?

Architecture explanation

Example:

Explain payment flow.

Modules:

repository_agent.py
query_planner.py
rag_engine.py

5. Node Model

Repository

(:Repository)

Directory

(:Directory)

File

(:File)

Properties:

name

path

language

size

Class

(:Class)

Properties:

name

visibility

Function

(:Function)

Properties:

name

parameters

return_type

Endpoint

(:Endpoint)

Properties:

route

method

Database

(:Database)

Properties:

type

name

Package

(:Package)

Properties:

name

version

6. Relationships

Structural

HAS_DIRECTORY

HAS_FILE

CONTAINS_CLASS

CONTAINS_FUNCTION

Code

CALLS

IMPORTS

INHERITS

IMPLEMENTS

USES

DEPENDS_ON

Database

READS_FROM

WRITES_TO

CONNECTS_TO

APIs

EXPOSES_ENDPOINT

HANDLES_REQUEST

AI Layer

DESCRIBED_BY

SIMILAR_TO

SEMANTICALLY_RELATED

7. Technology Stack

Backend

FastAPI

Language Parsers

Tree-sitter

Python AST

javalang

LLM

Claude

GPT-5

Gemini

Graph Database

Neo4j

Vector Database

Neo4j Vector Index

Orchestration

LangGraph

Cache

Redis

Queue

Celery + RabbitMQ

Storage

PostgreSQL

8. Folder Structure

Gitty-ai/

backend/

└─ **app.py**

└─ config/

└─ ingestion/

clone_repo.py

repo_loader.py

└─ parsers/

java_parser.py

python_parser.py

js_parser.py

ts_parser.py

└─ extractors/

dependency_extractor.py

relationship_extractor.py

llm_extractor.py

└─ graph/

graph_builder.py

cypher_generator.py

neo4j_client.py

└─ embeddings/

embedder.py

└─ chunking/

chunk_generator.py

└─ vector/

vector_index.py

└─ agents/

repository_agent.py

query_planner.py

```
|
|  └─ rag/
|
|  retriever.py
|
|  └─ api/
|
|  graph_routes.py
|  chat_routes.py
|
└─ utils/
```

9. Development Phases

Phase 1

Repository Ingestion

Goal:

Clone repositories and scan files.

Deliverables:

- GitHub loader
- File scanner
- Metadata extractor

Duration:

1 week

Phase 2

AST Parsing

Goal:

Understand code structure.

Deliverables:

- Java parser
- Python parser

- JS parser

Duration:

2 weeks

Phase 3

Knowledge Graph

Goal:

Populate Neo4j.

Deliverables:

- Nodes
- Relationships

Duration:

1 week

Phase 4

LLM Semantic Layer

Goal:

Infer hidden relationships.

Deliverables:

- Call chains
- DB usage
- Service dependencies

Duration:

2 weeks

Phase 5

Embeddings

Goal:

Semantic search.

Deliverables:

- **Chunk embeddings**
- **Vector index**

Duration:

1 week

Phase 6

Repository Agent

Goal:

Chat with repositories.

Deliverables:

- **LangGraph agent**
- **Cypher generation**
- **Retrieval**

Duration:

2 weeks

Phase 7

Visualization

Goal:

Interactive graph explorer.

Frontend:

React + Cytoscape.js

Duration:

2 weeks

Phase 8

Enterprise Features

Goal:

Production readiness.

Features:

- Incremental indexing
- Multi-user
- RBAC
- Background jobs
- GitHub webhooks

Duration:

3 weeks

10. Future Features

Architecture Diagram Generation

Generate:

Controller



Service



Repository



Database

Dead Code Detection

MATCH (f:Function)

```
WHERE NOT EXISTS{  
  MATCH (:Function)-[:CALLS]->(f)  
}
```

RETURN f

Impact Analysis

**If JwtService changes,
which files are affected?**

Automatic Documentation

Generate:

- **README**
 - **Sequence diagrams**
 - **API docs**
-

MCP Server

Expose repository graph to:

- **Claude Desktop**
 - **Cursor**
 - **VS Code**
 - **Windsurf**
-

Multi-Agent System

Agents:

- **Repository Agent**
- **Cypher Agent**
- **RAG Agent**
- **Documentation Agent**
- **Architecture Agent**
- **Refactoring Agent**